

Spark Performance Optimization Analysis for the Tweet Sentiment Pipeline

Muhui Wang

1. Introduction

This report analyzes the runtime behavior of the tweet sentiment pipeline implemented in Databricks for the final project. The pipeline contains four stages: `tweets_bronze`, `tweets_silver`, `tweets_gold`, and `gold_tweet_aggregations`. The purpose of this analysis is to identify where the pipeline spends most of its time, explain the likely causes, and propose practical optimization strategies based on the Spark UI and query metrics collected during a full refresh run.

Figure 1 shows that the full pipeline completed in **1h 8m 54s**. However, the runtime is highly uneven across stages. The `tweets_bronze` stage completed in **19s**, `tweets_silver` in **12s**, and `gold_tweet_aggregations` in about **4s**. In contrast, `tweets_gold` alone required **1h 4m 11s**. This makes the gold layer the clear performance bottleneck.

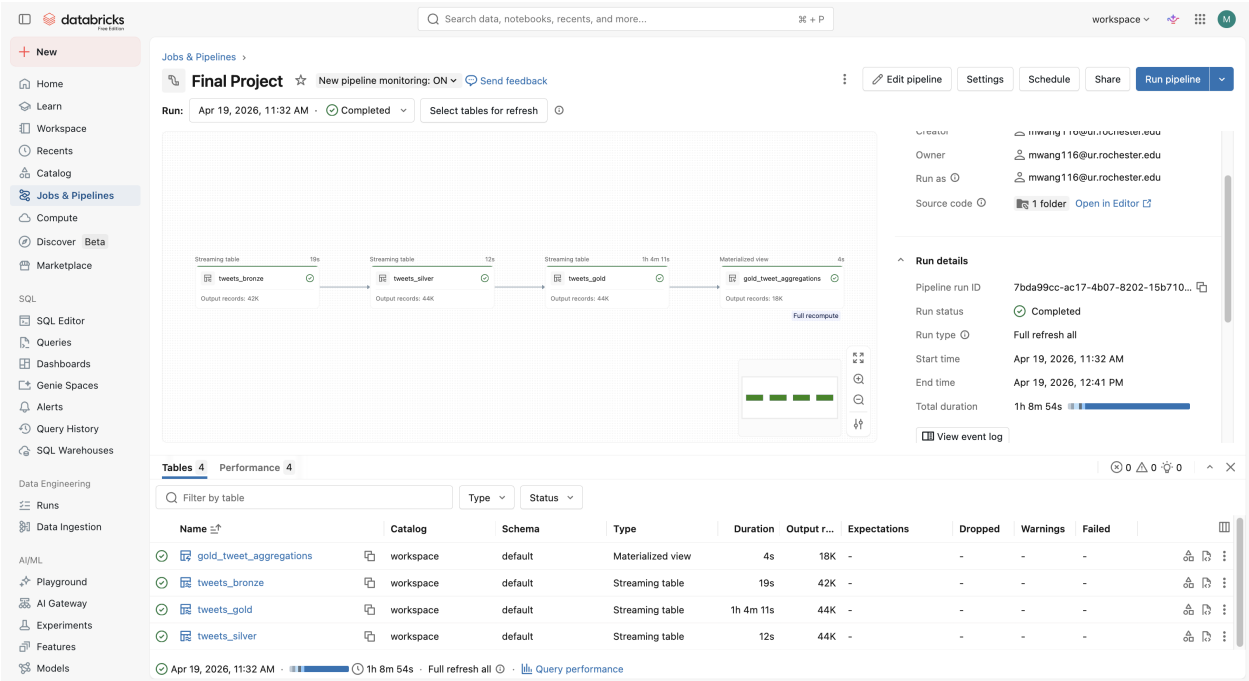


Figure 1: Full pipeline runtime overview. The overall runtime is dominated by the `tweets_gold` stage.

2. Primary Bottleneck: Gold-Layer Serialization and Inference Overhead

The strongest performance issue in this project appears in the `tweets_gold` stage. As shown in Figures 2 and 3, this stage processed only **43,658 rows**, read about **5.26 MB**, and wrote about **5.28 MB**, but still took more than one hour to finish. For a workload of this size, such a long runtime is disproportionate if the main cost were simply scanning or writing Delta data.

This suggests that the bottleneck is computational rather than I/O-driven. In this project, the gold layer applies sentiment prediction to the processed tweet data. Based on the query metrics, the most likely source of delay is Python-side execution, especially serialization and deserialization overhead between Spark and Python during model inference. The metrics also show that **Photon usage was 0%** for `tweets_gold`, which further supports the interpretation that this stage is dominated by execution patterns that do not benefit from Photon acceleration.

In other words, the main problem is not the size of the dataset itself, but the way sentiment inference is being executed on that data. This is the clearest bottleneck in the entire pipeline.

2.1. Optimization Recommendation

The most direct optimization would be to reduce Python-side inference overhead. If possible, row-wise Python UDF execution should be replaced by more efficient alternatives, such as:

- vectorized or batch-based inference,
- Spark-native transformations where feasible,
- reducing repeated Python-JVM boundary crossings,
- minimizing repeated model loading or repeated transformations before prediction.

2.2. Expected Impact

Because `tweets_gold` dominates total pipeline runtime, improving this stage would likely have the largest effect on overall performance. Even a moderate reduction in Python-side overhead could substantially shorten full refresh time.

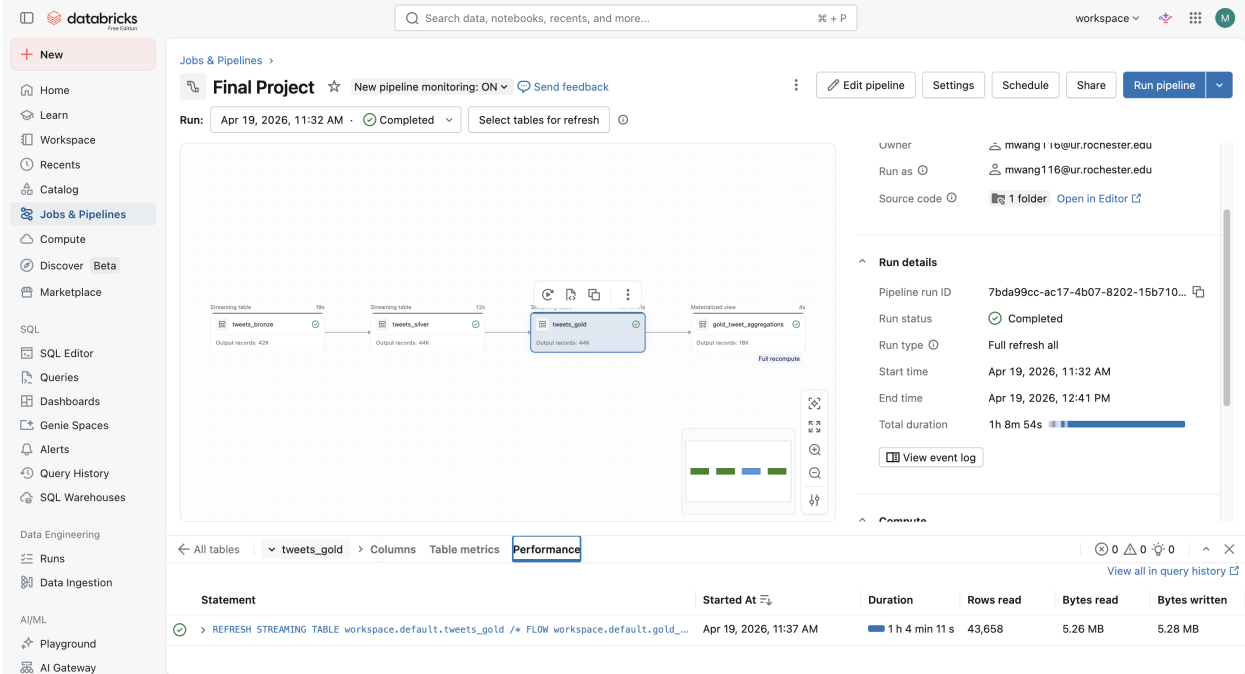


Figure 2: Performance summary for `tweets_gold`. The stage runtime is much larger than would be expected from the small data volume processed.

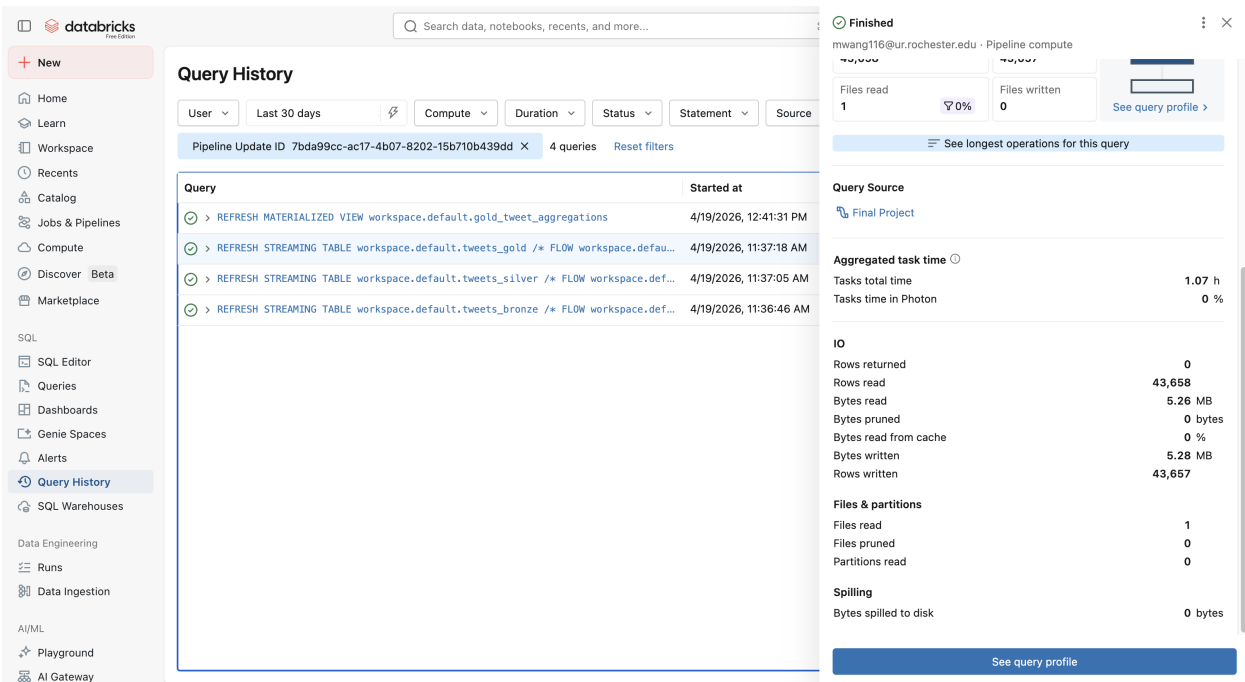


Figure 3: Query metrics for `tweets_gold` in Query History. The screenshots show 0 spill, small I/O, and no Photon acceleration, pointing toward computation and serialization overhead.

3. Secondary Costs in the Silver and Aggregation Logic

Although the gold layer is the dominant bottleneck, there are also several structural operations in the pipeline that may not dominate runtime in the current run but still introduce extra cost or future scalability risk.

3.1. Regex Extraction and `explode` in the Silver Layer

The silver layer extracts mentions from tweet text using regex and then uses `explode` to create one row per mention. This design is functionally correct, but it increases row cardinality because a single tweet can generate multiple rows after expansion. As a result, the silver layer not only performs text processing, but also enlarges the amount of data passed to later stages.

This does not appear to be the current bottleneck, since `tweets_silver` completed in only **12s**. However, it still increases downstream processing cost. In particular, it expands the input that must later be scored in `tweets_gold` and aggregated in `gold_tweet_aggregations`. So even if the regex and `explode` logic is not slow by itself, it acts as a workload multiplier for subsequent stages.

3.2. Shuffle and GroupBy Cost in the Application Layer

The final application-layer table, `gold_tweet_aggregations`, is created by grouping records by `mention` and computing positive, negative, and total counts. Any `groupBy` operation of this kind requires shuffle, since rows with the same key must be brought together before aggregation.

In this run, the aggregation stage completed quickly in about **4.29s**, which means shuffle is currently not a runtime bottleneck. Still, shuffle is an inherent cost of the design and should be considered part of the pipeline's performance profile. If the dataset grows, the cost of moving and grouping rows by `mention` may become more important.

3.3. Skew Risk After `explode`

A related issue is potential skew in the application layer. Mention frequencies are naturally uneven. Some user handles appear much more often than others. After the silver-layer `explode`, already frequent keys can become even more common, because each mention occurrence is materialized as a separate row. This means that the `groupBy(mention)` stage may become unbalanced if a few high-frequency keys dominate the data.

There is no strong evidence that skew is currently hurting performance, since the aggregation stage is still very fast and spill remains zero. Still, this is a meaningful design-level risk. If the data volume grows, skewed mention keys could lead to uneven task distribution and longer shuffle-heavy operations.

3.4. Optimization Recommendations

To address these issues, the following optimizations are reasonable:

- review mention frequency distribution before aggregation,
- avoid unnecessary expansion if downstream logic does not need every exploded row,
- tune shuffle partitions as the dataset grows,
- consider repartitioning on `mention` for larger workloads,
- apply skew-aware strategies only if strong key imbalance is observed in future runs.

3.5. Expected Impact

At the current scale, these changes would probably have less effect than fixing the gold-layer bottleneck. However, they would improve scalability and reduce the risk that regex expansion and `groupBy` operations become more expensive in larger future runs.

4. Spill Analysis

One important observation from the query metrics is that **bytes spilled to disk = 0** for both `tweets_gold` and `gold_tweet_aggregations`. This means the current run did not suffer from obvious memory pressure or shuffle spill.

This matters because it helps distinguish between possible causes of slow performance. If spill had been high, partition tuning or memory tuning would have been the first priority. Instead, the absence of spill suggests that the long runtime in `tweets_gold` is not caused by insufficient memory, but by compute-side overhead. Therefore, spill is not the primary issue in the current run.

4.1. Optimization Recommendation

Although spill is not currently a problem, partition tuning is still worth monitoring if the dataset grows. Future runs with larger data volume may require:

- reviewing the number of shuffle partitions,
- checking whether repartitioning is necessary before aggregation,
- re-evaluating spill metrics under higher-scale workloads.

4.2. Expected Impact

The immediate impact would likely be small for the current dataset, but this would help prevent future degradation if the pipeline is scaled up.

5. Storage Considerations

The Delta tables in this project are still relatively small. The metrics for `gold_tweet_aggregations` show less than **1 MB** read and less than **500 KB** written, and the gold stage also processes only a few megabytes of data. At this scale, storage layout is not the main bottleneck.

However, storage optimization is still relevant from a longer-term engineering perspective. If the project were extended to larger workloads or refreshed repeatedly over time, fragmented file layout and small files could eventually increase query cost for downstream analytics and dashboard refreshes.

5.1. Optimization Recommendation

For future workloads, the following Delta maintenance strategies would be reasonable:

- run `OPTIMIZE` on frequently queried Delta tables,
- consider `ZORDER BY mention` if the application layer is queried by `mention`,
- monitor file counts and file sizes after repeated refreshes.

5.2. Expected Impact

These changes are unlikely to make a large difference on the current dataset, but they would improve storage layout and help support larger-scale or more frequently refreshed versions of the pipeline.

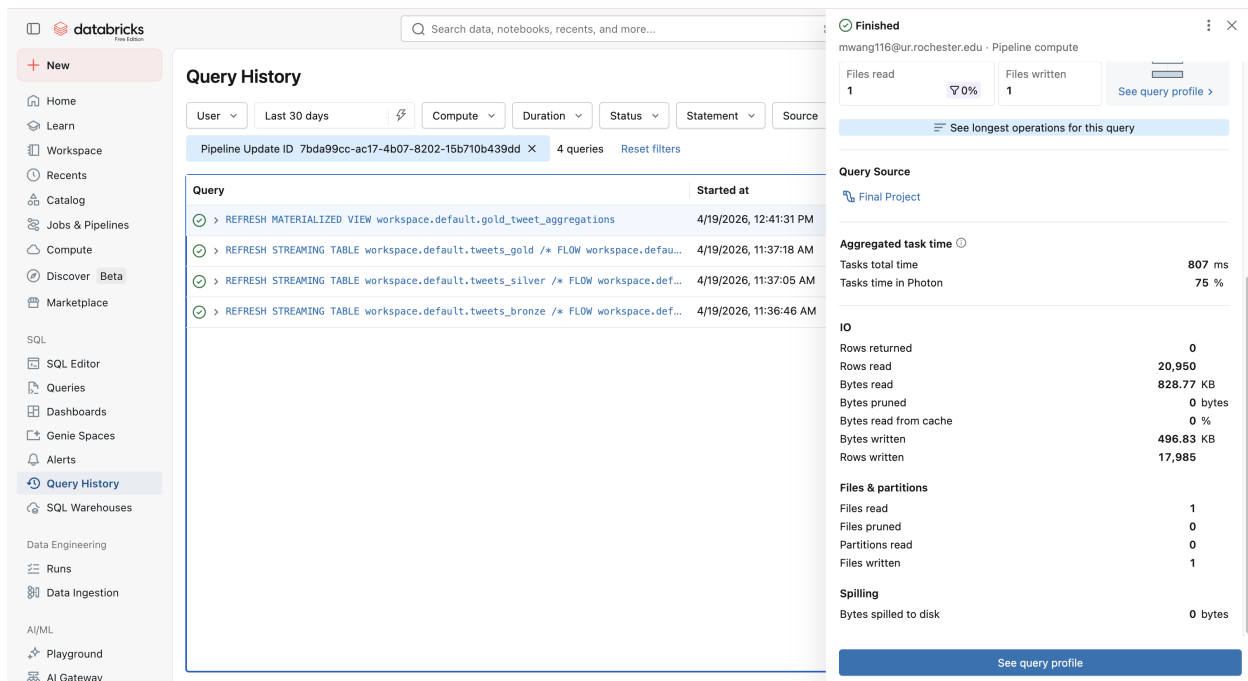


Figure 4: Query metrics for `gold_tweet_aggregations`. Aggregation is currently efficient, with low I/O, no spill, and relatively high Photon usage.

6. Conclusion

Based on the Spark UI and Databricks query metrics, the clearest bottleneck in this pipeline is the `tweets_gold` stage. Its runtime is much larger than expected relative to the amount of data processed, which strongly suggests Python-side inference and serialization overhead. This is the optimization target most likely to produce a meaningful reduction in total pipeline runtime.

At the same time, the project also contains several secondary performance concerns that are worth noting. The regex-based mention extraction and `explode` logic in the silver layer increases row cardinality and expands downstream workload. The application layer introduces shuffle through `groupBy(mention)`, and the uneven distribution of mentions may create skew risk if the pipeline is run on a larger dataset. Spill and storage are not major issues in the current run, but both should still be monitored as the project scales.

Overall, the most effective next step would be to improve the execution strategy of the gold layer while keeping an eye on row expansion, shuffle behavior, and future skew in the aggregation logic.